Active Record Associations

This guide covers the association features of Active Record.

Here are a few things you should know to make efficient use of Active Record associations in your Rails applications:

- Controlling caching
- Avoiding name collisions
- Updating the schema
- Controlling association scope
- Bi-directional associations

3.1 Controlling Caching

All of the association methods are built around caching, which keeps the result of the most recent query available for further operations. The cache is even shared across methods. For example:

```
customer.orders                 # retrieves orders from the database
customer.orders.size            # uses the cached copy of orders
customer.orders.empty?          # uses the cached copy of orders
```

But what if you want to reload the cache, because data might have been changed by some other part of the application? Just pass `true` to the association call:

```
customer.orders                 # retrieves orders from the database
customer.orders.size            # uses the cached copy of orders
customer.orders(true).empty?    # discards the cached copy of orders
                                # and goes back to the database
```

3.2 Avoiding Name Collisions

You are not free to use just any name for your associations. Because creating an association adds a method with that name to the model, it is a bad idea to give an association a name that is already used for an instance method of `ActiveRecord::Base`. The association method would override the base method and break things. For instance, `attributes` or `connection` are bad names for associations.

3.3 Updating the Schema

Associations are extremely useful, but they are not magic. You are responsible for maintaining your database schema to match your associations. In practice, this means two things, depending on what sort of associations you are creating. For `belongs_to` associations you need to create foreign keys, and for `has_and_belongs_to_many` associations you need to create the appropriate join table.

3.3.1 Creating Foreign Keys for `belongs_to` Associations

When you declare a `belongs_to` association, you need to create foreign keys as appropriate. For example, consider this model:

```
class Order < ActiveRecord::Base
  belongs_to :customer
end
```

This declaration needs to be backed up by the proper foreign key declaration on the orders table:

```
class CreateOrders < ActiveRecord::Migration
  def change
```